ARQUITECTURA DE DATOS · UNIDAD DE CALIDAD

Pruebas de software y garantía de calidad

Del aseguramiento de la calidad a la práctica: pytest para pruebas unitarias y de integración, SonarQube para seguridad y calidad de código.

```
$ pytest -v && sonar-scanner
```

RUTA DE LA SESIÓN

Agenda

01

Fundamentos de calidad

Por qué importa en datos, QA vs QC, norma ISO/IEC 25010.

02

Tipos y niveles de prueba

Pirámide de pruebas, funcionales y no funcionales, pruebas de datos.

03

Herramientas

pytest, cobertura, pruebas de seguridad, SonarQube y CI/CD.

04

Taller práctico

Pruebas con pytest y análisis con SonarQube sobre un mini-pipeline ETL.

CONTEXTO

La calidad en arquitectura de datos

Un defecto en una transformación no falla una sola vez: se replica en cada carga, cada reporte y cada modelo que consume esos datos.

Propagación

Los errores viajan aguas abajo por todo el pipeline.

Confianza

Las decisiones del negocio dependen de datos correctos.

Costo

Corregir en producción cuesta más que detectar en desarrollo: *shift-left*.

CONCEPTOS CLAVE

QA, QC y testing no son lo mismo

PROCESO · PREVENTIVO

Garantía de calidad (QA)

Define estándares, revisiones y métricas para que los defectos no aparezcan.

Ejemplo: acordar convenciones de código y un Quality Gate.

PRODUCTO · DETECTIVO

Control de calidad (QC)

Inspecciona el resultado para verificar que cumple los requisitos.

Ejemplo: revisar el informe de SonarQube antes de liberar.

ACTIVIDAD DENTRO DE QC

Pruebas (testing)

Ejecutar el software con casos definidos para encontrar defectos.

Ejemplo: correr la suite de pytest en cada commit.

MODELO DE CALIDAD DEL PRODUCTO

ISO/IEC 25010: qué es “calidad”

Azul = se trabaja en el taller

Adecuación funcional
Hace lo que debe, de forma correcta y completa. → pytest

Eficiencia de desempeño
Tiempos de respuesta y uso de recursos.

Compatibilidad
Coexiste e intercambia datos con otros sistemas.

Capacidad de interacción
Antes “usabilidad”: fácil de aprender y usar.

Fiabilidad
Funciona sin fallas y se recupera. → pytest, SonarQube

Seguridad
Protege datos y resiste ataques. → SonarQube

Mantenibilidad
Fácil de entender, modificar y probar. → SonarQube

Flexibilidad
Antes “portabilidad”: se adapta y escala.

Protección (safety)
Evita poner en riesgo personas o entorno.

NIVELES DE PRUEBA

La pirámide de pruebas



E2E / sistema · pocas

El flujo completo, como lo vive el usuario. Lentas y frágiles.

Integración · algunas

Módulos juntos: código + base de datos, archivos o API.

Unitarias · muchas

Una función aislada. Rápidas (milisegundos), baratas y precisas.

TIPOS DE PRUEBA

Qué pregunta responde cada prueba

Tipo	Pregunta	Herramientas	Clase
Unitaria	¿Esta función hace lo correcto?	pytest	Funcional
Integración	¿Los componentes funcionan juntos?	pytest + SQLite/Docker	Funcional
Sistema / E2E	¿El flujo completo cumple?	pytest, Playwright	Funcional
Aceptación	¿Cumple lo que pidió el usuario?	pytest-bdd, Behave	Funcional
Regresión	¿Sigue funcionando lo anterior?	Suite completa en CI	Funcional
Rendimiento	¿Responde con la carga esperada?	Locust	No funcional
Seguridad	¿Tiene vulnerabilidades?	SonarQube, Bandit	No funcional
Calidad de datos	¿Los datos cumplen las reglas?	pandera, GX	Datos

PRUEBAS EN ARQUITECTURA DE DATOS

Pruebas propias de un pipeline de datos

Dimensiones de calidad de datos

- **Completitud:** sin nulos indebidos
- **Exactitud:** refleja la realidad
- **Consistencia:** igual entre sistemas
- **Unicidad:** sin duplicados
- **Validez:** formato y dominio
- **Oportunidad:** llega a tiempo

Esquema y contratos

Columnas, tipos y restricciones que el origen se compromete a entregar.

Transformaciones

Cada función de limpieza o cálculo, con pruebas unitarias.

Carga e idempotencia

Ejecutar dos veces no duplica filas. Prueba de integración.

Reconciliación

Conteos y sumas de control iguales entre origen y destino.

HERRAMIENTAS · PYTEST

Pruebas unitarias con pytest

Descubrimiento
Archivos **test_*.py** y funciones **test_***.

assert simple
Sin métodos especiales: pytest explica el fallo.

@pytest.mark.parametrize
Una prueba, muchos casos de entrada.

pytest.raises
Verifica que se lance la excepción esperada.

Fixtures
Preparan y limpian datos o recursos.

```python
# tests/test_transform.py
import pytest
from etl.transform import normalizar_email

def test_normaliza_mayusculas_y_espacios():
    assert normalizar_email("  Ana@Mail.COM ") == "ana@mail.com"

@pytest.mark.parametrize("entrada", ["", "sin-arroba", None])
def test_email_invalido_lanza_error(entrada):
    with pytest.raises(ValueError):
        normalizar_email(entrada)
```

```
$ pytest -v
4 passed in 0.03s
```

HERRAMIENTAS · PYTEST

Pruebas de integración con pytest

Verifican que **leer → transformar → cargar** funcione junto, con recursos reales pero controlados.

sqlite3 en memoria

Base de datos real, desechable y rápida.

tmp_path

Fixture de pytest: carpeta temporal para el CSV.

yield

Lo que sigue al yield limpia el recurso.

```python
# tests/test_integracion.py
import sqlite3, pytest
from etl.pipeline import ejecutar_pipeline
@pytest.fixture
def conexion():
    con = sqlite3.connect(":memory:")
    yield con
    con.close()
def test_pipeline_carga_clientes(conexion, tmp_path):
    csv = tmp_path / "clientes.csv"
    csv.write_text("id,email\n1, Ana@Mail.com\n2,luis@mail.com\n")
    ejecutar_pipeline(csv, conexion)
    filas = conexion.execute(
        "SELECT email FROM clientes ORDER BY id").fetchall()
    assert filas == [("ana@mail.com",), ("luis@mail.com",)]
```

HERRAMIENTAS · PYTEST-COV

Cobertura de código con pytest-cov

```
$ pytest --cov=etl --cov-report=term-missing \
         --cov-report=xml
```

Name	Stmts	Miss	Cover	Missing
etl/transform.py	12	1	92%	16
etl/pipeline.py	10	0	100%	
TOTAL	22	1	95%	

Salida ilustrativa. El archivo coverage.xml lo leerá SonarQube.

Qué mide

Qué líneas (y ramas, con --cov-branch) se ejecutaron durante las pruebas.

Qué no mide

Si las aserciones son buenas. 100% de cobertura no significa cero defectos.

Meta de referencia

≥ 80% en código nuevo: condición del Quality Gate “Sonar way”.

PRUEBAS NO FUNCIONALES

Pruebas de seguridad del código

SAST

Análisis estático

Revisa el código fuente sin ejecutarlo.

SonarQube, Bandit

DAST

Análisis dinámico

Ataca la aplicación en ejecución.

OWASP ZAP

SCA

Dependencias

Busca librerías con vulnerabilidades conocidas.

pip-audit, Dependabot

Secrets

Credenciales

Detecta claves y tokens dentro del repositorio.

gitleaks, SonarQube

Riesgos típicos en código de datos (OWASP Top 10)

Inyección SQL al armar consultas con f-strings · credenciales en el código · uso de **eval** sobre datos externos · manejo de errores que oculta fallas · dependencias desactualizadas.

HERRAMIENTAS · SONARQUBE

SonarQube y el Quality Gate

Fiabilidad

Bugs: código que fallará o se comportará mal.

Seguridad

Vulnerabilidades que un atacante puede explotar.

Security Hotspots

Código sensible que una persona debe revisar.

Mantenibilidad

Code smells y deuda técnica estimada.

Cobertura

Importada desde coverage.xml de pytest-cov.

Duplicación

Bloques de código repetidos.

QUALITY GATE “SONAR WAY”

Aprobado o rechazado, sobre el código nuevo:

- 0 issues nuevos
- Hotspots revisados
- Cobertura ≥ 80%
- Duplicación ≤ 3%

INTEGRACIÓN CONTINUA

Todo junto en un pipeline de CI/CD

1 **Commit / PR**
git push

→

2 **Pruebas**
pytest unitarias + integración

→

3 **Cobertura**
coverage.xml

→

4 **Análisis**
sonar-scanner

→

5 **Quality Gate**
¿Aprobado?

Aprobado → despliegue
El cambio pasa a la rama principal y a producción.

Rechazado → se detiene
Se corrige y se vuelve al paso 1. El defecto no llega a producción.

```
assert calidad == proceso
```

La calidad no se inspecciona al final: se construye en cada commit.

TALLER PRÁCTICO

Taller: calidad en un mini-pipeline ETL

Objetivo: aplicar pruebas unitarias y de integración con pytest, y pruebas de seguridad y calidad de código con SonarQube, sobre un pipeline que lee clientes desde un CSV, los limpia y los carga en SQLite.

1	2	3	4	5
Entorno	**Unitarias**	**Integración**	**SonarQube**	**Defectos**
venv, pytest, Docker	transform.py y cobertura	CSV → SQLite	Primer análisis	Encontrar y corregir

Duración: [__ horas] **Modalidad:** parejas **Requisitos:** Python 3.11+, Docker, editor de código

TALLER · PASO 1

Paso 1 · Preparar el entorno

ESTRUCTURA DEL PROYECTO

```
taller-calidad/
├── etl/
│   ├── __init__.py
│   ├── transform.py
│   ├── pipeline.py
│   └── reportes.py  # paso 5
├── tests/
│   ├── __init__.py
│   ├── test_transform.py
│   └── test_integracion.py
├── requirements.txt
└── sonar-project.properties
```

TERMINAL

```
# crear y activar el entorno virtual
python -m venv .venv
source .venv/bin/activate
# Windows: .venv\Scripts\activate

# instalar dependencias
pip install pytest pytest-cov
pip freeze > requirements.txt

# verificar herramientas
pytest --version
docker --version
```

TALLER · PASO 2

Paso 2 · El código bajo prueba

```python
# etl/transform.py
import re
PATRON = re.compile(r"^[\w.+-]+@[\w-]+\.[\w.]+$")

def normalizar_email(email):
    if not isinstance(email, str) or not email.strip():
        raise ValueError("email vacío")
    limpio = email.strip().lower()
    if not PATRON.match(limpio):
        raise ValueError(f"email inválido: {email}")
    return limpio

def calcular_total(precio, cantidad, descuento=0.0):
    if precio < 0 or cantidad < 0 or not 0 <= descuento <= 1:
        raise ValueError("parámetros fuera de rango")
    return round(precio * cantidad * (1 - descuento), 2)
```

Tareas

1. Copia este archivo en etl/transform.py.
2. Antes de escribir pruebas, lista en papel 3 casos válidos y 3 inválidos por función.
3. Incluye casos límite: descuento 0 y 1, cantidad 0, espacios y mayúsculas en el email.

TALLER · PASO 2

Paso 2 · Escribir pruebas unitarias

```python
# tests/test_transform.py
import pytest
from etl.transform import normalizar_email, calcular_total

@pytest.mark.parametrize("entrada, esperado", [
    ("  Ana@Mail.COM ", "ana@mail.com"),
    ("luis.p+datos@uni.edu.co", "luis.p+datos@uni.edu.co"),
])
def test_normalizar_email_valido(entrada, esperado):
    assert normalizar_email(entrada) == esperado

@pytest.mark.parametrize("entrada", ["", "   ", "sin-arroba", None])
def test_normalizar_email_invalido(entrada):
    with pytest.raises(ValueError):
        normalizar_email(entrada)
```

Tareas

1. Escribe al menos 4 pruebas para calcular_total: descuento, negativos, descuento > 1 y redondeo.
2. Ejecuta: **pytest -v**
3. Mide: **pytest --cov=etl --cov-report=term-missing**
4. Meta: transform.py ≥ 90% y todas las pruebas en verde.

TALLER · PASO 3

Paso 3 · Integración del pipeline

```python
# etl/pipeline.py
import csv
from etl.transform import normalizar_email

def ejecutar_pipeline(ruta_csv, con):
    con.execute("CREATE TABLE IF NOT EXISTS clientes "
                "(id INTEGER PRIMARY KEY, email TEXT)")
    with open(ruta_csv, encoding="utf-8") as f:
        filas = [(int(r["id"]), normalizar_email(r["email"]))
                 for r in csv.DictReader(f)]
    con.executemany(
        "INSERT OR REPLACE INTO clientes VALUES (?, ?)", filas)
    con.commit()
    return len(filas)
```

Tareas

1. Implementa pipeline.py.
2. Usa la prueba de la diapositiva 10 como tests/test_integracion.py.
3. Agrega una prueba de idempotencia: ejecutar dos veces deja 2 filas, no 4.
4. Agrega un CSV con un email inválido: decide qué debe pasar y pruébalo.

TALLER · PASO 4

Paso 4 · Analizar con SonarQube

```
# sonar-project.properties (en la raíz del proyecto)
sonar.projectKey=taller-calidad
sonar.sources=etl
sonar.tests=tests
sonar.python.version=3.11
sonar.python.coverage.reportPaths=coverage.xml
```

```
# 1. Levantar SonarQube → http://localhost:9000 (usuario admin / clave admin)
docker run -d --name sonarqube -p 9000:9000 sonarqube:community
# 2. Generar la cobertura en XML
pytest --cov=etl --cov-report=xml
# 3. Analizar con el token creado en My Account › Security
docker run --rm --network host -e SONAR_HOST_URL="http://localhost:9000" \
  -e SONAR_TOKEN="$SONAR_TOKEN" -v "$(pwd):/usr/src" sonarsource/sonar-scanner-cli
```

TALLER · PASO 5

Paso 5 · Encontrar y corregir defectos

```python
# etl/reportes.py — defectos INTENCIONALES para el análisis
import sqlite3
DB_PASSWORD = "admin123"

def buscar_cliente(con, email):
    consulta = f"SELECT * FROM clientes WHERE email = '{email}'"
    return con.execute(consulta).fetchall()

def evaluar_regla(expresion, fila):
    return eval(expresion)

def total(filas):
    try:
        return sum(f[1] for f in filas)
    except:
        pass
```

Tareas

1. Agrega reportes.py y vuelve a analizar.
2. Ubica en SonarQube cada hallazgo y su tipo: hotspot, bug o code smell.
3. Corrige: variable de entorno, consulta con “?”, sin eval, excepción específica.
4. Prueba que buscar_cliente resiste ' OR '1'='1.
5. Re-analiza hasta aprobar el Quality Gate.

EVALUACIÓN DEL TALLER

Entregables y rúbrica de evaluación

Criterio	Evidencia esperada	Peso
Pruebas unitarias	≥ 8 pruebas con parametrize y pytest.raises	25%
Integración	Pipeline CSV → SQLite e idempotencia probada	25%
Cobertura	≥ 80% del paquete etl (reporte adjunto)	15%
SonarQube	Capturas antes y después; Quality Gate aprobado	25%
Informe breve	Hallazgos, correcciones y reflexión (1–2 páginas)	10%

Entrega: repositorio Git con el código, las pruebas y el informe · Pesos propuestos, ajustables por el docente.

CIERRE

¿Preguntas?

Documentación de pytest
docs.pytest.org

Documentación de SonarQube
docs.sonarsource.com

OWASP Top 10
owasp.org/www-project-top-ten

ISO/IEC 25010
iso25000.com